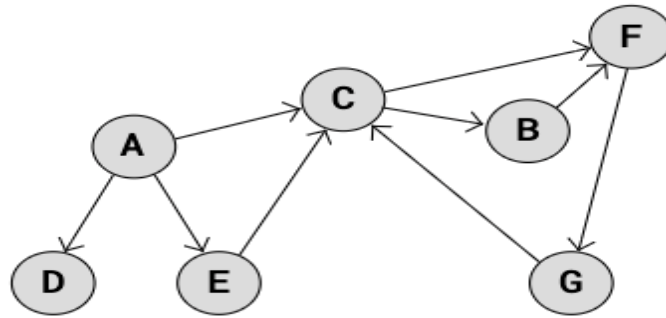


# Introduction to Graphs

- A Graph (G) is a collection of Vertices (V) and Edges (E).  
Used to represent networks, maps, & relationships.  
{Notation:  $G = (V, E)$ }
- A Graph is a non-linear data structure that consists of vertices (nodes) and edges.



- A vertex, also called a node, is a point or an object in the Graph, and an edge is used to connect two vertices with each other.

# Types of Graphs

- 1. Undirected Graph – edges have no direction
- 2. Directed Graph (Digraph) – edges have direction
- 3. Weighted Graph – edges have weights
- 4. Unweighted Graph – no weights on edges
- 5. Simple Graph – no loops or multiple edges
- 6. Multigraph – may contain multiple edges

# Graph Terminology

- Degree – number of edges incident on vertex
- Path – sequence of connected vertices
- Cycle – closed path
- Connected Graph – path exists between all vertices
- Complete Graph – every vertex connected to all others

# Graph Representation

- A Graph representation tells us how a Graph is stored in memory.
  - 1) Adjacency Matrix
  - 2) Adjacency List
- The Adjacency Matrix is a 2D array (matrix) where each cell on index  $(i,j)$  stores information about the edge from vertex  $i$  to vertex  $j$ .
- Below is a Graph with the Adjacency Matrix representation next to it.

Example –

## Adjacency Matrix Representation

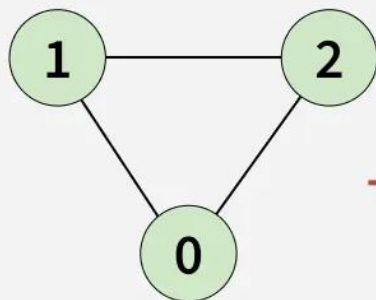
An adjacency matrix is a way of representing a graph as a boolean matrix of (0's and 1's).

Let's assume there are  $n$  vertices in the graph So, create a 2D matrix  $\text{adjMat}[n][n]$  having dimension  $n \times n$ .

*If there is an edge from vertex  $i$  to  $j$ , mark  $\text{adjMat}[i][j]$  as 1.*

*If there is no edge from vertex  $i$  to  $j$ , mark  $\text{adjMat}[i][j]$  as 0.*

### Representation of Undirected Graph as Adjacency Matrix:



Undirected Graph



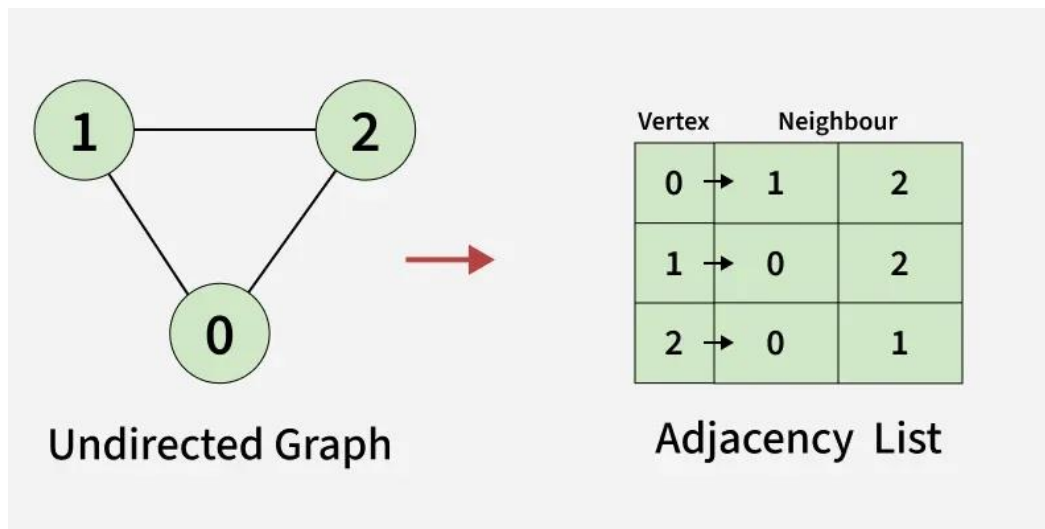
|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 2 | 1 | 1 | 0 |

Adjacency Matrix

# Adjacency List Representation

An array of Lists is used to store edges between two vertices. The size of array is equal to the number of vertices (i.e,  $n$ ). Each index in this array represents a specific vertex in the graph. The entry at the index  $i$  of the array contains a linked list containing the vertices that are adjacent to vertex  $i$ . Let's assume there are  $n$  vertices in the graph So, create an array of list of size  $n$  as `adjList[n]`.

- *`adjList[0]` will have all the nodes which are connected (neighbour) to vertex 0.*
- *`adjList[1]` will have all the nodes which are connected (neighbour) to vertex 1 and so on.*
- **Representation of Undirected Graph as Adjacency list:**



# Graph Traversal

- Two main traversal techniques:
  - 1) Breadth First Search (BFS) – Level order using queue
  - 2) Depth First Search (DFS) – Deep traversal using recursion or stack

# Breadth First Search (BFS)

## Algorithm:

- 1. Start from a source vertex
- 2. Visit all adjacent vertices
- 3. Move to next level

## Applications:

- Shortest path in unweighted graphs
- Social network analysis



# Depth First Search (DFS)

- Algorithm:
  1. Start from a source vertex
  2. Visit unvisited adjacent vertex recursively
  3. Backtrack when needed
- Applications:
  - Topological sorting
  - Cycle detection
  - Pathfinding problems

# Applications of Graphs

- Social Networks (Facebook, LinkedIn)
- Computer Networks (Routing)
- Google Maps (Shortest path)
- Compiler Design (Flow graphs)
- AI and Machine Learning (Graph Neural Networks)

# Conclusion

- Graphs are essential for modeling real-world problems.
- Representation choice affects performance.
- Core to applications in networking, AI, and optimization.